Project Report

Publishing House Management System

Submitted by

Karan Kumar
Matriculation No. 7175598

Asif Majeed
Matriculation No. 7175625

For the subject

Software Architecture and Methodologies

Submitted to

Prof. Enrico Vicario

Università Degli Studi di Firenze

Contents

1 Application Context and Problem Statement 1
1.1 Application Context 1
1.2 Target Users and Key Needs 1
1.3 Problem Statement 1
2 Requirements Analysis 2
2.1 User Roles 2
2.2 Functional Requirements 2
2.2.1 Must Have (Critical Functionality) 3
2.2.2 Should Have (Important but not Vital) 4
2.2.3 Won't Have (Out of Scope) 4
2.3 Non-Functional Requirements 5
2.3.1 Architectural & Technological Constraints 5
2.3.2 Security 5
2.3.3 Dependability & Data Integrity 6
2.3.4 Usability & Interface 6
3 Conceptual Domain Model 7
3.1 Class Diagram 7
3.2 Entity Descriptions 8
3.2.1 Core Entities 8
3.2.2 User & Personnel Entities 9
4 Use Case Analysis 10
4.1 Use Case Diagram 10
4.2 Detailed Use Case Specifications 11
4.2.1 UC0: Login (Authentication) 11
4.2.2 UC1: Manage Publications 11
4.2.3 UC2: Manage Author Registry 13
4.2.4 UC3: Manage Personnel 14

5 Architectural Considerations 15
5.1 Frontend Layer (Angular) 15
5.2 Backend Layer (Java / Jakarta EE) 15
5.3 Database Layer (PostgreSQL) 16
5.4 Deployment Diagram 16
6 Implementation Details 18
6.1 Backend Implementation (Jakarta EE) 18
6.1.1 Project Structure 18
6.1.2 Security & Authentication 19
6.1.3 REST API Design 20
6.2 Frontend Implementation (Angular) 20
6.2.1 Technology Stack 20
6.2.2 Project Structure 20
6.2.3 Integration with Backend 21
6.3 Development Environment 21

1 Application Context and Problem Statement

1.1 Application Context

This project focuses on the design and development of a web-based management system tailored for an **international publishing house**. The operative domain is characterized by a high volume of data and a heterogeneous catalogue of products. The publishing house manages a vast array of literary works, including textbooks, novels, graphic novels, and essays.

The domain model is complex due to the nature of the publishing industry. A single publication may exist in various editions, each associated with distinct pricing models, metadata, and unique **ISBN (International Standard Book Number)** codes. Additionally, the system must account for the relationships between publications and a registry of authors, tracking the history of works associated with specific writers.

1.2 Target Users and Key Needs

The Web Application is designed to serve internal stakeholders within the enterprise, specifically focusing on the operational management of the catalogue and resources.

- **Administrative Back-Office Personnel:** These users form the operational core of the system. Their primary need is efficiency in data entry and record maintenance. They require centralized tools for:
  - **Catalogue Management:** Inserting new records and updating details for pre-existing publications.
  - **Registry Management:** Editing the authors' registry and maintaining historical data of their publications.
  - **Personnel Management:** Maintaining records for internal staff members such as editor managers and reviewers.

1.3 Problem Statement

Prior to the proposed solution, the internal management process of the publishing house was characterized by **fragmentation and decentralization**. The organization relied on a disjointed ecosystem of tools, including unconnected text-processing software, spreadsheets, and manual paper records. This resulted in significant inefficiencies, data redundancy, and a high risk of inconsistency during data entry.

The goal of this project is to resolve these issues by **digitalizing and centralizing** these core functions into a single Web Application using a RESTful architectural style, ensuring data consistency and streamlined operations for back-office personnel.

2 Requirements Analysis

This section outlines the Software Requirements Specification (SRS) for the Publishing House Management System. It serves as a comprehensive guide for the development team, detailing the specific behaviors, functions, and constraints of the system.

2.1 User Roles

The system identifies four distinct user roles, each designed with specific permissions to enforce a secure, hierarchical workflow:

- **Admin (System Administrator):** This role possesses the highest level of privilege. The Admin has full control over the entire system, including complete management of employees, authors, and publications. This role is responsible for system maintenance and personnel onboarding.
- **Manager (Team Lead):** Designed for operational supervisors, this role has full control over the core business data (authors and publications). While they can create, read, and update employee records to manage their team's details, they **cannot** delete personnel records, ensuring that employment history is preserved and preventing accidental data loss.
- **Editor (Content Creator):** This role focuses strictly on the publishing content. Editors can manage authors and publications (Add, Edit, View) to keep the catalogue up to date. However, to ensure privacy and security, they have **no access** to the personnel (employee) module and cannot see user data.
- **User (Viewer/Intern):** A basic read-only role intended for junior staff or general reference. Users can only view authors and publications to retrieve information. They have no write access and are strictly blocked from personnel management or editing features to maintain data integrity.

2.2 Functional Requirements

The functional requirements are categorized using MoSCoW method (Must Have, Should Have, Could Have, Won't Have) to ensure the development team focuses on critical

deliverables first. They are further grouped by functional module for clarity.

2.2.1 Must Have (Critical Functionality)

These requirements are mandatory for the system to be operational and meet the core objectives of the assignment.

Authentication Module

- **FR1**: The system SHALL allow users to securely perform login using a unique email and password. The system must validate these credentials against the database and provide feedback in case of authentication failure.
- **FR2**: The system SHALL automatically redirect users to a role-specific dashboard upon successful login. For example, the interface must dynamically hide the Employee management tab for Editors and Users to prevent unauthorized navigation.

Personnel Management Module

- **FR3**: The system SHALL allow the **Admin** to perform full CRUD (Create, Read, Update, Delete) operations on Employee records. This includes managing sensitive fields such as names, contact information, and system role assignments.
- **FR4**: The system SHALL allow the **Manager** to Create, Read, and Update Employee records, enabling them to correct details or add new staff members as needed.
- **FR5**: The system SHALL strictly prevent the **Manager** from deleting Employee records. This restriction is critical to preserve historical data and ensures that only Administrators can remove users from the system.
- **FR6**: The system SHALL strictly prevent **Editors** and **Users** from accessing, viewing, or interacting with the Personnel Management module. Any attempt to access these resources via direct URL entry must be blocked.

Catalogue (Publications) Module

- **FR7**: The system SHALL allow **Admins** and **Managers** to perform full CRUD operations on Publication records. This includes managing complex data such as Title, multiple ISBNs, Publication Date, Price, and Edition details.
- **FR8**: The system SHALL allow **Editors** to Create, Read, and Update Publication records, ensuring the catalogue remains current with the latest book releases and edition updates.

- **FR9**: The system SHALL allow the **User** role to only View the list of publications and their specific details, ensuring they can reference catalogue data without the risk of accidental modification.

Author Management Module

- **FR10**: The system SHALL allow **Admins** and **Managers** to perform full CRUD operations on Author records, including managing biographical data and linking authors to their respective publications.
- **FR11**: The system SHALL allow **Editors** to Create, Read, and Update Author records, essential for correcting author profiles or adding new contributors to the database.
- **FR12**: The system SHALL allow the **User** role to only View the list of authors and their details, providing read-only access to the author registry.

2.2.2 Should Have (Important but not Vital)

These features should be included if time and resources permit, as they significantly improve usability and workflow efficiency.

- **FR13**: The system SHALL allow users to search for specific publications by filtering the list based on Title or ISBN. This feature is crucial for quickly locating items in a large catalogue.
- **FR14**: The system SHALL allow users to filter the author list by name, improving the speed of data retrieval.
- **FR15**: The system SHALL provide clear visual feedback (e.g., "Operation Successful" toast notifications or distinct error messages) after any CRUD operation to confirm the action's status to the user.

2.2.3 Won't Have (Out of Scope)

These features are explicitly excluded from the current prototype to manage project scope, though they may be considered for future iterations.

- **FR16**: The system WILL NOT include a sales statistics dashboard or reporting module for this version. The focus remains on data management rather than business intelligence.

- **FR17**: The system WILL NOT support self-registration (Sign Up) for new users. To maintain tight security, all user accounts must be created manually by an Admin or Manager.

2.3 Non-Functional Requirements

This subsection defines the constraints and quality attributes of the system, categorized by their nature.

2.3.1 Architectural & Technological Constraints

These constraints are mandated by the project assignment and dictate the technical stack and design patterns.

- **NFR1 - RESTful Architecture:** The system SHALL be designed according to the REST (Representational State Transfer) architectural style. It must adhere to stateless communication constraints, where every client request contains all necessary information for the server to fulfill it.
- **NFR2 - Backend Implementation:** The Backend module SHALL be implemented using **Java** and strict **Jakarta EE** specifications (JAX-RS for web services, JPA for persistence).
- **NFR3 - Frontend Implementation:** The Frontend module SHALL be developed as a Single Page Application (SPA) using the **Angular** framework and **TypeScript**.
- **NFR4 - Data Exchange Format:** The system SHALL use **JSON** (JavaScript Object Notation) as the exclusive format for data exchange between the Frontend and Backend.
- **NFR5 - Data Persistence:** The system SHALL persist data using a relational database (SQL) managed via a **JPA** (Jakarta Persistence API) implementation, abstracting direct SQL queries.

2.3.2 Security

These requirements ensure the system protects data and restricts access to authorized personnel only.

- **NFR6 - Server-Side Authorization:** The system SHALL enforce Role-Based Access Control (RBAC) checks at the API level (Backend). It is insufficient to merely hide UI elements; the server must reject unauthorized HTTP requests (e.g., a 'User' attempting to DELETE an employee) with a '403 Forbidden' status.

- **NFR7 - Stateless Authentication:** The system SHOULD utilize a token-based authentication mechanism (e.g., JWT or basic bearer tokens) to maintain the stateless nature of the REST architecture, avoiding server-side session storage.

2.3.3 Dependability & Data Integrity

These attributes focus on the reliability of the data and the system's ability to handle errors.

- **NFR8 - Referential Integrity:** The system SHALL enforce strict referential integrity within the database. For example, a Publication cannot be created for an Author that does not exist, and an Author cannot be deleted if they have linked publications that rely on their record (unless a cascading delete strategy is explicitly defined).
- **NFR9 - Standardized Error Handling:** The system SHALL return appropriate HTTP Status Codes (e.g., '200 OK', '201 Created', '404 Not Found', '500 Internal Server Error') to indicate the outcome of operations clearly to the client.

2.3.4 Usability & Interface

These requirements ensure the application is accessible and easy to use for the target audience.

- **NFR10 - Responsive Design:** The system SHALL ensure the User Interface (UI) uses a responsive layout (e.g., via CSS Flexbox or Grid) that adapts to standard desktop and tablet screen resolutions.
- **NFR11 - Minimal Latency:** The system SHOULD ensure that standard CRUD operations (such as fetching the list of books) complete within a reasonable time frame (e.g., under 2 seconds) to ensure a smooth workflow for back-office personnel.

3 Conceptual Domain Model

This section describes the data entities involved in the system and their relationships. The domain model is designed to support the core business processes of the publishing house, including catalogue management, author tracking, and personnel administration.

3.1 Class Diagram

The following UML Class Diagram visualizes the structure of the database and the relationships between entities.

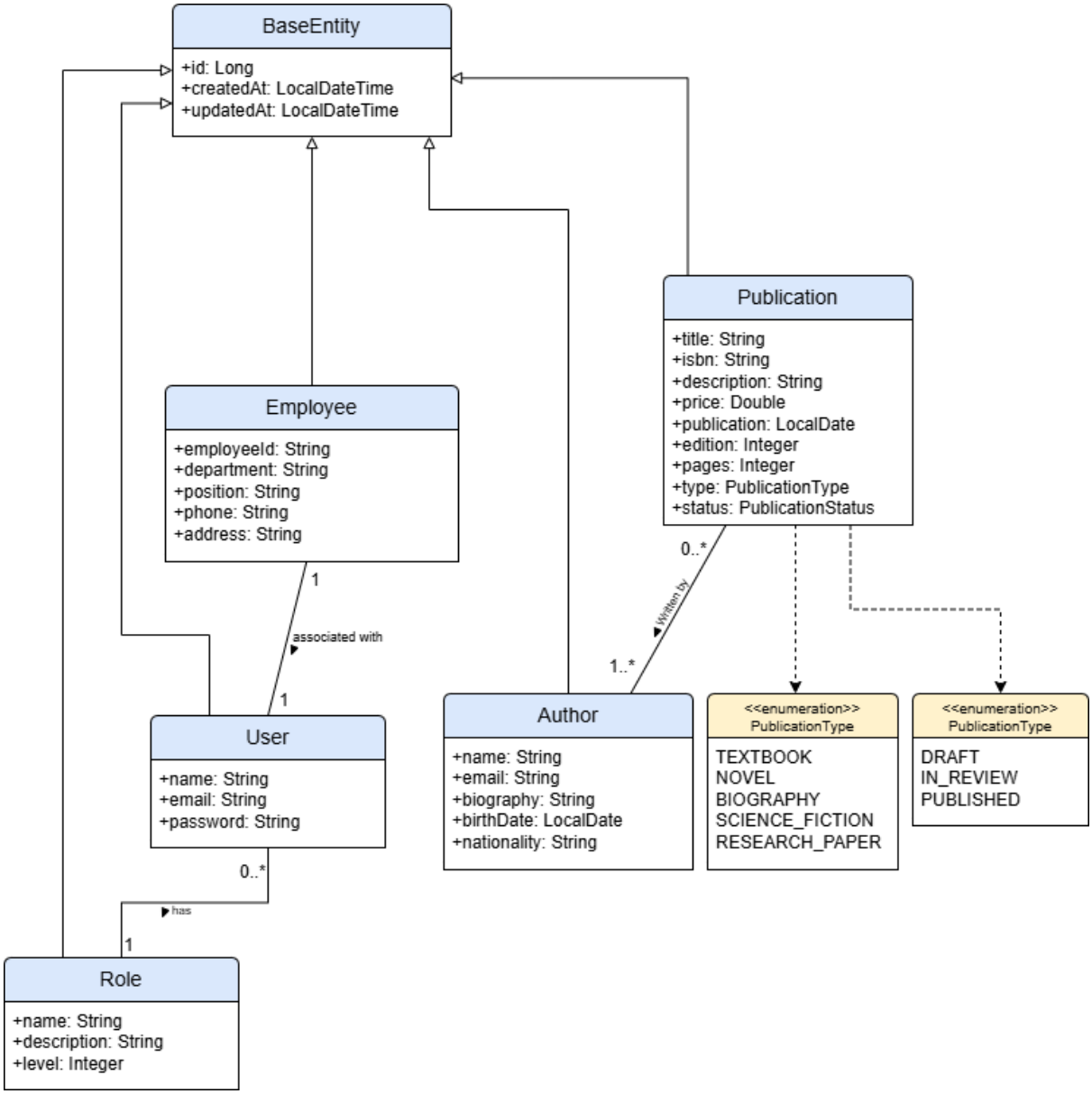

Figure 1: UML Class Diagram of the Publishing House Management System

3.2 Entity Descriptions

The system is built upon a generic **BaseEntity** class, which acts as a superclass for all persistent objects.

- **Attributes:** `id` (Primary Key), `createdAt`, `updatedAt`.
- **Behavior:** It utilizes JPA Lifecycle callbacks (`@PrePersist`, `@PreUpdate`) to automatically manage timestamps whenever a record is created or modified.

3.2.1 Core Entities

1. Publication (PublicationModel)
Represents a book or literary work in the catalogue. This is the central entity of the domain.

- **Attributes:**
  - `title`: The full title of the work.
  - `isbn`: A unique 17-character International Standard Book Number.
  - `description`: A text summary or synopsis.
  - `price`: The commercial cost of the publication.
  - `publicationDate`: The official release date.
  - `edition`: An integer indicating the version (defaults to 1).
  - `pages`: The total number of pages.
  - `type`: Categorizes the work via `PublicationType` (e.g., TEXTBOOK, NOVEL).
  - `status`: Tracks the editorial stage via `PublicationStatus` (e.g., DRAFT, PUBLISHED).

2. Author (AuthorModel)
Represents a writer who creates publications.

- **Attributes:**
  - `name`: The full name of the author.
  - `email`: A unique contact email.
  - `biography`: A short text describing the author's career.
  - `birthDate`: The date of birth.
  - `nationality`: The country of origin.

3.2.2 User & Personnel Entities

3. User (UserModel)

Represents an individual who can log in to the system. This entity handles authentication data.

- **Attributes:**
  - `name`: The display name of the user.
  - `email`: Unique email address used as a username.
  - `password`: The encrypted credential.
- **Relationships:** Many-to-One with **Role**: Every user is assigned exactly one permission role.

4. Role (RoleModel)

Defines a set of permissions within the application.

- **Attributes:**
  - `name`: The unique name of the role (e.g., "ADMIN", "EDITOR").
  - `level`: A numeric value representing the hierarchy of privilege.
  - `description`: Details about what this role is allowed to do.

5. Employee (EmployeeModel)

Extends the user concept with specific employment details. Not all Users are Employees, but every Employee must have a User account.

- **Attributes:**
  - `employeeId`: A unique internal ID string (e.g., "EMP-001").
  - `department`: The operational unit (e.g., "Editorial", "Warehouse").
  - `position`: The job title.
  - `phone`: Contact number.
  - `address`: Physical mailing address.
- **Relationships:** One-to-One with **User**: Links the professional record to the login credentials.

4 Use Case Analysis

This section analyzes primary interactions within the Publishing House Management System, bridging Section 2 requirements and Section 3 conceptual models to demonstrate business value.

Based on requirements, three core Use Cases illustrate the system's capabilities: **Manage Publications**, **Manage Authors**, and **Manage Personnel**. These cover essential CRUD operations and relationship management mandated by the assignment.

4.1 Use Case Diagram

The following diagram illustrates the actors and their interactions with the system modules.

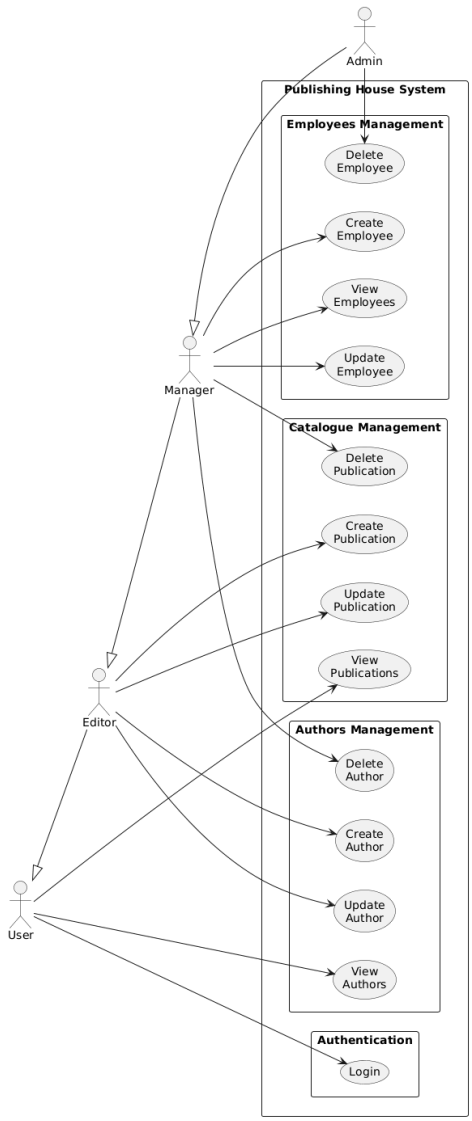

Figure 2: Use Case Diagram of the Publishing House Management System

4.2 Detailed Use Case Specifications

4.2.1 UC0: Login (Authentication)

This fundamental use case allows users to authenticate with the system to access protected resources based on their role.

- **Use Case Name:** Login
- **Authorized Actors:** All Users (Admin, Manager, Editor, User)
- **Resources Involved:** `UserModel`, `JWTUtil`
- **Associated REST Endpoints:**
  - `POST /api/auth/login`
- **User Story:** "As a registered user, I want to log in with my credentials so that I can access the system features corresponding to my role."

Main Flow:

1. **Actor** navigates to the login page.
2. **Actor** enters email and password.
3. **System** verifies credentials against the database.
4. **System** generates a signed JWT token containing the user's role.
5. **System** returns the token to the client.

Alternative & Exception Flows:

- **Invalid Credentials:** If the email or password does not match, the System returns a `401 Unauthorized` status with the message "Invalid email or password."

4.2.2 UC1: Manage Publications

This central use case involves maintaining the digital book catalogue, including product details, editions, and status workflows (Draft to Published).

- **Use Case Name:** Manage Publications
- **Authorized Actors:** Editor, Manager, Admin

- **Resources Involved:** `PublicationModel`, `PublicationDAO`
- **Associated REST Endpoints:**
  - `GET /api/publications`
  - `POST /api/publications`
  - `PATCH /api/publications/{id}`
  - `DELETE /api/publications/{id}`
- **User Story:** "As an Editor or Manager, I want to add and update book details so the catalogue remains accurate for sales and inventory."

Main Flow (Create Publication):

1. **Actor** clicks "Add New" in the publications view.
2. **System** displays the `PublicationModel` form (Title, ISBN, Price, Edition, Pages, Type, Status, Description, Publication Authors).
3. **Actor** fills fields, selecting strictly typed `PublicationType` and `PublicationStatus` (e.g., `DRAFT`).
4. **Actor** selects the publication author*s* from the list of present authors
5. **Actor** submits.
6. **System** validates input (checking ISBN uniqueness).
7. **System** persists the entity via `PublicationDAO`.
8. **System** returns `201 Created` with the JSON resource and redirects to the list.
9. **System** shows the toast "Publication created successfully!"

Alternative & Exception Flows:

- **Duplicate ISBN:** If an ISBN exists, System catches `DataIntegrityViolationException`, returning `409 Conflict`. UI warns: "A publication with ISBN 9780306406157 already exists."
- **Validation Failure:** Missing mandatory fields prevents submission and highlights errors.

4.2.3 UC2: Manage Author Registry

This use case ensures accurate attribution by managing author histories and linking `Authors` to `Publications` for bibliographies.

- **Use Case Name:** Manage Author Registry
- **Authorized Actors:** Editor, Manager, Admin
- **Resources Involved:** `AuthorModel`, `PublicationModel` (Linkage), `AuthorRepository`
- **Associated REST Endpoints:**
  - `GET /api/authors`
  - `POST /api/authors`
  - `PATCH /api/authors/{id}`
- **User Story:** "As an Editor, I want to maintain an author registry and link books to track contributor portfolios."

Main Flow (Update Author & Link Book):

1. **Actor** accesses "Author Registry."
2. **System** lists `AuthorModel` entities.
3. **Actor** selects an author to edit.
4. **System** loads current details (Name, Email, Bio).
5. **Actor** updates bio and links a new `Publication` from a list.
6. **System** updates the `Author` entity and `authors_publications` join table.
7. **System** updates `updatedAt` timestamp and confirms save.

Alternative & Exception Flows:

- **Author Not Found:** Accessing a non-existent ID returns `404 Not Found`.

4.2.4 UC3: Manage Personnel

Distinct from other cases, this involves simultaneous management of `User` (auth) and `Employee` (business data) entities for workforce administration.

- **Use Case Name:** Manage Personnel
- **Authorized Actors:** Admin (Full Access), Manager (Partial Access: cannot delete)
- **Resources Involved:** `UserModel`, `EmployeeModel`, `RoleModel`
- **Associated REST Endpoints:**
  - `GET /api/employees`
  - `POST /api/employees`
  - `DELETE /api/employees/{id}` (Restricted to Admin)
- **User Story:** "As an Admin, I want to onboard employees by creating accounts and roles for appropriate system access."

Main Flow (Onboard New Employee):

1. **Admin** selects "Add New" in Employee view
2. **System** requests split data:
   - **User Credentials:** Name, Email, Password, Role.
   - **Employee Details:** ID, Department, Position, Phone, Address.
3. **Admin** submits data.
4. **System** executes atomic transaction:
   - Persists `UserModel` with encrypted password.
   - Creates `EmployeeModel` linked via `user_id` foreign key.
5. **System** confirms creation; user can log in immediately.

Alternative & Exception Flows:

- **Unauthorized Deletion: Manager** attempting deletion triggers `403 Forbidden` due to lack of `DELETE` permission.
- **Email Conflict:** Registering an existing email rolls back the transaction and displays an error.

5 Architectural Considerations

The application follows a layered architecture that separates concerns between the frontend, backend, and database to enhance scalability, maintainability, and clear responsibility boundaries. Each layer can be deployed on a different machine, adhering to a standard 3-tier architecture.

5.1 Frontend Layer (Angular)

The frontend is implemented using **Angular 20**, which runs in the user's browser and communicates with the backend solely through RESTful HTTP requests. It handles user interaction, view rendering, client-side routing, and form validation. The frontend sends and receives JSON payloads for resources such as publications, authors, and employees. It is styled using **Tailwind CSS** for a responsive design and manages authentication tokens (JWTs) securely in memory to authenticate API requests.

5.2 Backend Layer (Java / Jakarta EE)

The backend is developed using **Jakarta EE 10**, deployed on a **WildFly** application server. It is structured into modular layers to ensure separation of concerns:

1. **REST Layer (rest):** Defines HTTP endpoints using **JAX-RS**. Each endpoint maps to a specific resource and HTTP verb (e.g., `GET /api/publications`, `POST /api/employees`).
   - **Security Enforcement:** Instead of standard container security, the application uses a custom annotation `@JWTTokenNeeded`. This annotation is applied to methods or classes (e.g., `@JWTTokenNeeded(roles = {"ADMIN", "MANAGER"})`) to enforce role-based access control directly at the resource level.
   - **CORS Handling:** A custom `CORSFilter` is implemented to handle Cross-Origin Resource Sharing, allowing the Angular frontend to communicate securely with the backend during development and production.
2. **Service Layer (service):** Encapsulates the application's business logic. It validates operations (e.g., ensuring a Manager cannot delete an Employee record) and manages transaction boundaries using CDI.
3. **DAO Layer (repository):** Manages persistence via the **JPA** (Jakarta Persistence API) specification. It interacts with the PostgreSQL database using `EntityManager`, utilizing typed queries for pagination, filtering, and custom lookups.

4. **Model / DTO Layers:**
    - **Entities:** Represent persisted domain objects annotated with `@Entity` (e.g., `PublicationModel`, `EmployeeModel`). They use a `BaseEntity` superclass to handle common lifecycle events like `createdAt` timestamps using `@PrePersist`.
    - **DTOs:** Data Transfer Objects are used to expose data safely to the frontend, preventing over-fetching and hiding internal details like password hashes.

Security Implementation

- **Authentication:** Stateless authentication is implemented using **JWT (JSON Web Tokens)**. A custom `JWTUtil` class generates tokens upon login and validates them for subsequent requests.
- **Authorization:** The `AuthenticationFilter` intercepts HTTP requests annotated with `@JWTTokenNeeded`. It parses the Bearer token, extracts the user's role, and verifies if it matches the required permissions before the request reaches the endpoint.
- **Password Storage:** User passwords are never stored in plain text. They are hashed using the **BCrypt** algorithm (via `PasswordUtil`) with a cost factor of 12 to ensure robust protection against brute-force attacks.

5.3 Database Layer (PostgreSQL)

The **PostgreSQL** DBMS persists application data using JPA entities. The schema is managed by the Hibernate provider bundled with WildFly. It stores normalized tables for `users`, `employees`, `publications`, and `authors`. Foreign key constraints (e.g., between `employees` and `users`) enforce referential integrity, ensuring consistent data across the system.

5.4 Deployment Diagram

The following diagram illustrates the physical deployment of the software artifacts.

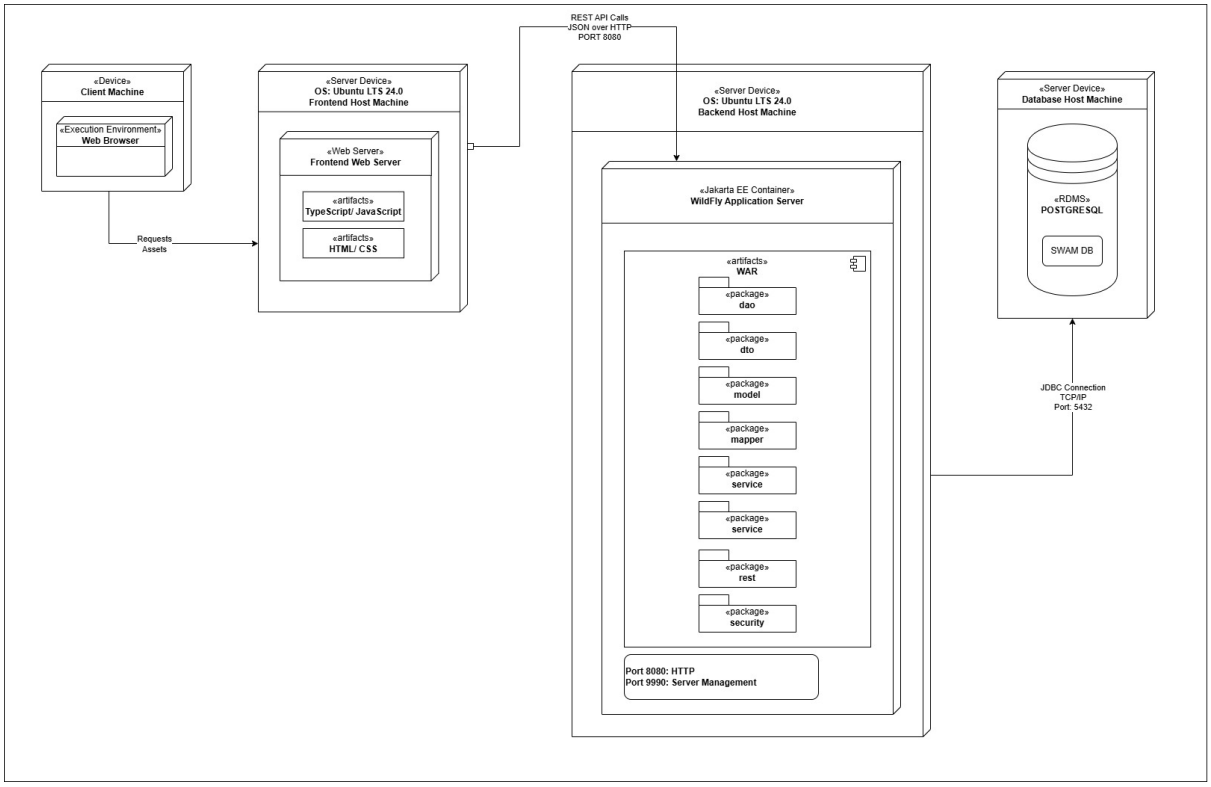

Figure 3: Deployment Diagram of the Publishing House Management System

6 Implementation Details

This section details the implementation of the Publishing House Management System, developed using **Java 21 lts** with **Jakarta EE 10** for the backend and **Angular 20** for the frontend.

6.1 Backend Implementation (Jakarta EE)

The backend is a Maven-based project deployed on a **WildFly** application server. It adheres to a strictly layered architecture to separate concerns between data access, business logic, and API exposure.

6.1.1 Project Structure

The codebase is organized into the following packages under `org.swam.publishing_house`:

- `model`: Contains JPA Entities mapped to the database.
  - Core entities: `PublicationModel`, `AuthorModel`, `EmployeeModel`, `UserModel`, `RoleModel`.
  - `BaseEntity.java`: Superclass handling `id`, `createdAt`, and `updatedAt` with `@PrePersist`/`@PreUpdate` lifecycle callbacks.
  - Enums: `PublicationType`, `PublicationStatus`.
- `dao`: Data Access Objects handling persistence logic.
  - Interfaces: `PublicationDAO`, `AuthorDAO`, `UserDAO`, `EmployeeDAO`.
  - Implementation: `impl/` package (e.g., `RoleDAOImpl`) using `EntityManager` for database interactions.
- `service`: Business logic layer (CDI Beans).
  - Includes `PublicationService`, `AuthorService`, `EmployeeService`, `UserService`, `AuthService`.
  - Orchestrates calls to DAOs and manages transactional integrity.
- `rest`: JAX-RS resources defining the API endpoints.
  - Controllers: `PublicationRest`, `AuthorRest`, `EmployeeRest`, `UserRest`, `AuthRest`.
- `dto`: Data Transfer Objects for API payloads.

  - Organized by domain: `dto/publication/`, `dto/author/`, `dto/employee/`, `dto/user/`.
  - Common DTOs: `ApiResponseDTO`, `PagedResult` for pagination.

- `mapper`: Utilities for transforming Entities to DTOs and vice-versa.
  - Classes: `PublicationMapper`, `AuthorMapper`, `UserMapper`, etc.

- `security`: Security infrastructure components.
  - Components: `JWTTokenNeeded` (Annotation), `AuthenticationFilter`, `JWTUtil`, `PasswordUtil`, `CORSFilter`, `SecurityContext`.

- `util`: Helper classes.
  - `PasswordUtil` (BCrypt hashing), `PaginationUtil`, `ErrorHandlerUtil`.

6.1.2 Security & Authentication

Security is stateless and token-based, implemented manually without heavy frameworks like Spring Security.

1. **Authentication:**
   - Users login via `AuthRest`. `AuthService` verifies credentials using `PasswordUtil` (which implements BCrypt with a cost factor of 12).
   - Upon success, `JWTUtil` generates a signed **JWT** (using `io.jsonwebtoken`) containing the user's email and role.

2. **Authorization (RBAC):**
   - Protected endpoints are annotated with the custom name-binding annotation `@JWTTokenNeeded`.
   - Example in `EmployeeRest`:

```
@POST
@JWTTokenNeeded(roles = { ADMIN ,  MANAGER })
public Response createEmployee(@Valid
   EmployeeCreateRequestDTO request) { ... }
```

3. **Request Filtering:**

- `AuthenticationFilter` intercepts secure requests, validates the token signature via `JWTUtil`, and ensures the user's role matches the roles defined in the annotation.
- `CORSFilter` manages Cross-Origin Resource Sharing headers (`Access-Control-Allow-Origi` `*`) to support the Angular development server.

6.1.3 REST API Design

The API follows REST principles using JAX-RS (`@Path`, `@GET`, `@POST`, `@DELETE`).

- **Input Validation:** Uses Jakarta Validation (`@Valid`) on DTOs.
- **Response Format:** Wraps responses in `ApiResponseDTO` for consistent structure (success/error messages).
- **Pagination:** Implemented in DAOs and Services using `PagedResult` and `PaginationUtil` to handle large datasets efficiently.

6.2 Frontend Implementation (Angular)

The frontend is an **Angular 20** Single Page Application (SPA).

6.2.1 Technology Stack

- **Core:** Angular 20, TypeScript.
- **Styling: Tailwind CSS 4** (configured via `@tailwindcss/postcss`).
- **State/Async:** RxJS (~7) for observable-based API interaction.
- **UI Utilities:** `ngx-toastr` for notifications.

6.2.2 Project Structure

The application logic is modularized into three main directories within `src/app`:

- `core`: Contains singleton services and structural elements that are instantiated once.
  - **Constants:** `api.constants.ts`, `roles.constants.ts`.
  - **Guards:** `auth.guard.ts` (protects routes), `role.guard.ts` (checks user permissions).
  - **Interceptors:** `auth.interceptor.ts` (attaches Bearer tokens to requests).
  - **Services:** `auth.service.ts` (login/logout), `permission.service.ts`.

- `features`: Contains the functional modules of the application.
  - `auth`: Handles login logic (`login.component.ts`).
  - `author`: Manages author registry (`author-list`, `author-view`, `author-edit`).
  - `employees`: Manages staff records (`employee-list`, `employee-edit`).
  - `publications`: Manages the book catalogue (`publication-list`, `publication-edit`).
  - `dashboard`: The main landing view after login.
- `shared`: Contains reusable UI components and utilities.
  - **UI Components:** `button`, `input-field`, `pagination`, `main-layout`.
  - **Utils:** `date.utils.ts`, `form.utils.ts`, `validation.utils.ts`.

6.2.3 Integration with Backend

The application uses Angular's `HttpClient` to communicate with the Jakarta EE backend. The `AuthInterceptor` ensures that every HTTP request directed to the API includes the valid JWT in the Authorization header, maintaining the stateless security context required by the server.

6.3 Development Environment

- **Server:** WildFly (provisioned via `wildfly-maven-plugin`).
- **Database: PostgreSQL** (configured via `run.sh` with `swamdb`).
- **Build Automation:** `run.sh` script sets environment variables (`POSTGRESQL_URL`, `JWT_SECRET`) and executes the Maven build (`mvn clean package wildfly:package`).